

Practical No: 06

Aim:

To sort documents in ascending or descending order and limit the number of query results using MongoDB.

Theory:

Sorting and limiting results are essential operations in database management, especially when dealing with large datasets. MongoDB provides powerful and efficient methods for arranging documents in a desired order and restricting the number of results returned. These operations help users organize data, improve readability, and optimize application performance.

The **sort()** method in MongoDB arranges the result set based on one or more fields. Sorting can be performed in two directions:

- **Ascending order (1)** — lowest to highest
- **Descending order (-1)** — highest to lowest

For example, sorting by age: 1 arranges students from youngest to oldest, while sorting by name: -1 arranges students' names from Z to A. Sorting is particularly useful for ranking, alphabetical lists, chronological arrangements, or when selecting top or bottom entries.

The **limit()** function restricts the number of documents returned by a query. This is especially helpful when the collection is large, and only a subset of results is required, such as displaying the top 5 students or only the first few matching records. Limiting results also reduces network overhead and improves UI responsiveness in applications

Commands:

```
db.students.find().sort({ age: 1 })
db.students.find().sort({ name: -1 }).limit(2)
```

Example: Sorting in MongoDB

```
db.documents.find().sort({ date: 1 }) // Ascending order (oldest to newest)
```

1. For descending order (newest to oldest):

```
db.documents.find().sort({ date: -1 }) // Descending order (newest to oldest)
```

2. Limiting the Number of Results:

```
db.documents.find()
  .sort({ date: -1 }) // Sort in descending order (newest first)
  .limit(5);         // Limit the result to 5 documents
```

3. Combining Sort and Limit:

```
db.documents.find()  
  .sort({ date: -1 }) // Sort by 'date' in descending order  
  .limit(10);        // Return only 10 documents
```

4. Multiple Sort Criteria:

```
db.documents.find()  
  .sort({ date: -1, title: 1 }) // Sort by 'date' descending and 'title' ascending  
  .limit(5);                    // Limit the result to 5 documents
```

5. Example with Skip and Limit for Pagination:

```
db.documents.find()  
  .skip(10) // Skip the first 10 documents (for page 2)  
  .limit(10) // Limit the result to 10 documents (page size)  
  .sort({ date: -1 }); // Sort by 'date' descending
```

Expected Output:

Sorted list of documents with limited count.

Viva Questions:

1. What is sorting in MongoDB?

Sorting in MongoDB is the process of arranging query results in a specific order based on one or more fields.

2. Which method is used for sorting documents in MongoDB?

The `sort()` method is used to sort documents in MongoDB.

3. What values are used for ascending and descending order in `sort()`?

1 is used for ascending order and -1 is used for descending order.

4. Why is sorting important in database applications?

Sorting helps organize data, improve readability, and present information in a meaningful order such as rankings or alphabetical lists.

Conclusion:

In this practical, sorting and limiting operations were successfully implemented using MongoDB's `sort()` and `limit()` methods. The experiment demonstrated how documents can be arranged in ascending or descending order and how the number of query results can be controlled efficiently. These operations are essential for handling large datasets, improving application performance, and presenting meaningful and organized data in real-world applications such as dashboards, reports, and analytics systems.

